

Java Programming: From C

Chapter: 04. Managing Data with Collections

Prepared by: Chunghyun Lim (Matriculated in 2019)

Last Updated: June 2026



목차

1. Collection
2. Collection: ArrayList
3. Collection: HashSet
4. Collection: HashMap

1. Collection

컬렉션이란?

- 컬렉션(collection)은 여러 값을 하나로 묶어 저장하고 관리하기 위한 도구이다.
- 컬렉션은 여러 값을 담아 한꺼번에 관리할 수 있도록 표준 라이브러리에서 제공하는 자료구조(data structure)들의 모음이다.

```
ArrayList<String> todoItems = new ArrayList<>();
todoItems.add("Buy milk");
todoItems.add("Submit assignment");
todoItems.add("Call mom");

for (int i = 0; i < todoItems.size(); ++i) {
    System.out.println("- " + todoItems.get(i));
}
```

1. Collection

컬렉션에 저장할 수 있는 값

- Java의 컬렉션은 기본 자료형을 직접 다루지 않고 참조 자료형을 사용한다.
- 컬렉션에서 기본 자료형에 해당하는 값을 다룰 때는 대응되는 래퍼 자료형(wrapper type)을 사용한다.
 - 기본 자료형과 래퍼 자료형은 서로 다른 자료형이지만 Java가 값을 넣거나 꺼내는 과정에서 자동으로 변환(박싱/언박싱)해준다.

기본 자료형	래퍼 자료형
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

1. Collection

컬렉션 선택 기준

- 컬렉션은 저장 방식과 접근 방법에 따라 여러 종류로 나뉜다.
- 어떤 컬렉션을 사용할지는 데이터를 어떻게 다룰지에 따라 결정된다.
- 컬렉션을 선택할 때 고려할 사항은 다음과 같다.
 - 접근 기준
 - 정해진 index로 접근하는가?
 - 임의의 key로 접근하는가?
 - 데이터 접근 패턴
 - 데이터를 처음부터 끝까지 순회하는가?
 - 데이터를 자주 추가하거나 삭제하는가?
 - 컬렉션의 종류에 따라 메모리 사용량과 성능이 달라질 수 있지만 여기서는 세부적인 내용을 다루지 않는다.

2. Collection: ArrayList

ArrayList란?

- ArrayList는 같은 자료형의 데이터를 순서대로 저장하며 index로 데이터에 접근하는 컬렉션이다.
- 내부적으로 배열을 사용하지만 용량을 자동으로 관리해주는 가변 길이 배열 기반 컬렉션이다.

```
ArrayList<자료형> 변수명 = new ArrayList<>();
```

```
// import 영역
```

```
import java.util.ArrayList;
```

```
ArrayList<Integer> searchResultIds = new ArrayList<>();
```

2. Collection: ArrayList

요소 추가하기: add()

```
boolean add(자료형 element);  
void add(int index, 자료형 element);
```

- index를 지정하지 않은 경우 ArrayList의 맨 뒤에 요소(element)를 추가한다.
 - 요소가 정상적으로 추가된 경우 true를 반환한다.
- index를 지정한 경우 ArrayList의 해당 위치에 요소를 삽입한다.
 - 삽입 위치 뒤의 기존 요소들은 뒤로 한 칸씩 이동한다.
 - index가 유효하지 않은 경우 예외가 발생한다.

```
searchResultIds.add(101);  
searchResultIds.add(101);  
searchResultIds.add(309);  
searchResultIds.add(1, 150);
```

```
{> searchResultIds = {ArrayList@854}
```

```
{> 0 = {Integer@857} 101
```

```
{> searchResultIds = {ArrayList@854}
```

```
{> 0 = {Integer@857} 101
```

```
{> 1 = {Integer@857} 101
```

```
{> searchResultIds = {ArrayList@854}
```

```
{> 0 = {Integer@857} 101
```

```
{> 1 = {Integer@857} 101
```

```
{> 2 = {Integer@861} 309
```

```
{> searchResultIds = {ArrayList@854}
```

```
{> 0 = {Integer@857} 101
```

```
{> 1 = {Integer@864} 150
```

```
{> 2 = {Integer@857} 101
```

```
{> 3 = {Integer@861} 309
```

2. Collection: ArrayList

요소 조회하기: get()

자료형 `get(int index);`

- 지정한 위치(index)에 저장된 요소(element)를 반환한다.
- index가 유효하지 않은 경우 예외가 발생한다.

```
int selectedResultId = searchResultIds.get(1);
```

```
{ } searchResultIds = {ArrayList@854}  
{ } 0 = {Integer@857} 101  
{ } 1 = {Integer@864} 150  
{ } 2 = {Integer@857} 101  
{ } 3 = {Integer@861} 309  
1 selectedResultId = 150
```


2. Collection: ArrayList

요소 교체하기: set()

자료형 set(**int** index, 자료형 element);

- 지정한 위치(index)에 저장된 요소(element)를 새 요소로 교체한다.
- 해당 위치에 교체되기 전 저장되어 있던 요소를 반환한다.
- index가 유효하지 않은 경우 예외가 발생한다.

```
searchResultIds.set(1, 210);
```

```
{ } searchResultIds = {ArrayList@854}  
{ } 0 = {Integer@857} 101  
{ } 1 = {Integer@871} 210  
{ } 2 = {Integer@857} 101  
{ } 3 = {Integer@861} 309
```

2. Collection: ArrayList

요소 제거하기: remove()

자료형 remove(int index);

- 지정한 위치(index)에 저장된 요소(element)를 ArrayList에서 제거한다.
- 제거된 요소를 반환한다.
- 해당 위치 뒤에 있던 요소들은 내부 배열의 연속적인 구조를 유지하기 위해 한 칸씩 앞으로 이동한다.
- index가 유효하지 않은 경우 예외가 발생한다.

```
searchResultIds.remove(0);
```

```
{ } searchResultIds = {ArrayList@854}  
{ } 0 = {Integer@871} 210  
{ } 1 = {Integer@857} 101  
{ } 2 = {Integer@861} 309
```

2. Collection: ArrayList

요소 개수 확인하기: size()

```
int size();
```

- ArrayList에 현재 저장된 요소(element)의 개수를 반환한다.
 - 저장 가능한 요소의 개수가 아니라 현재 저장된 요소의 개수를 반환한다.

```
int searchResultCount = searchResultIds.size();
```

```
{> searchResultIds = {ArrayList@854}
  {> 0 = {Integer@871} 210
  {> 1 = {Integer@857} 101
  {> 2 = {Integer@861} 309
  [1] searchResultCount = 3
```

[다양한 ArrayList 메서드 보기 -> Java 17 API Specification]

3. Collection: HashSet

HashSet이란?

- HashSet은 중복을 허용하지 않는 값들의 집합을 저장하는 컬렉션이다.
- 같은 값이 이미 저장되어 있는지를 확인하며 데이터를 관리한다.

```
HashSet<자료형> 변수명 = new HashSet<>();
```

```
// import 영역
```

```
import java.util.HashSet;
```

```
HashSet<String> processedUsers = new HashSet<>();
```

3. Collection: HashSet

요소 추가하기: add()

```
boolean add(자료형 element);
```

- HashSet에 요소(element)를 추가한다.
- 요소가 새로 추가된 경우 true를 반환한다.
- 동일한 요소가 이미 저장되어 있는 경우 false를 반환한다.

```
boolean isNewUser;
```

```
isNewUser = processedUsers.add("Alice");
```

```
isNewUser = processedUsers.add("Alice");
```

```
isNewUser = processedUsers.add("Bobby");
```

```
{ } processedUsers = {HashSet@854}
```

```
{ } 0 = "Alice"
```

```
1 isNewUser = true
```

```
{ } processedUsers = {HashSet@854}
```

```
{ } 0 = "Alice"
```

```
1 isNewUser = false
```

```
{ } processedUsers = {HashSet@854}
```

```
{ } 0 = "Alice"
```

```
{ } 1 = "Bobby"
```

```
1 isNewUser = true
```

3. Collection: HashSet

요소 저장 여부 확인하기: contains()

```
boolean contains(자료형 element);
```

- 지정한 요소(element)가 HashSet에 저장되어 있는지 여부를 반환한다.

```
boolean isProcessed;
```

```
isProcessed = processedUsers.contains("Bobby");
```

```
{ } processedUsers = {HashSet@854}  
{ } 0 = "Alice"  
{ } 1 = "Bobby"  
1 isProcessed = true
```

3. Collection: HashSet

요소 제거하기: remove()

```
boolean remove(자료형 element);
```

- 지정한 요소(element)를 HashSet에서 제거한다.
- 해당 요소가 저장되어 있어 제거에 성공한 경우 true를 반환한다.
- 해당 요소가 저장되어 있지 않은 경우 false를 반환한다.

```
processedUsers.remove("Alice");
```

```
{ } processedUsers = {HashSet@854}  
{ } 0 = "Bobby"
```

3. Collection: HashSet

요소 개수 확인하기: size()

```
int size();
```

- HashSet에 현재 저장된 요소(element)의 개수를 반환한다.
 - 저장 가능한 요소의 개수가 아니라 현재 저장된 요소의 개수를 반환한다.

```
int processedUserCount = processedUsers.size();
```

```
{ } processedUsers = {HashSet@854}  
{ } 0 = "Bobby"  
1 processedUserCount = 1
```

[다양한 HashSet 메서드 보기 -> Java 17 API Specification]

4. Collection: HashMap

HashMap이란?

- HashMap은 key-value 쌍으로 데이터를 저장하며 key를 통해 value에 접근하는 컬렉션이다.
- key는 중복될 수 없으며 하나의 key는 항상 하나의 value에만 매핑된다.
 - 서로 다른 key는 같은 value에 매핑될 수 있다.
- index가 아닌 key 값을 기준으로 데이터를 관리한다.

```
HashMap<key 자료형, value 자료형> 변수명 = new HashMap<>();
```

```
// import 영역
```

```
import java.util.HashMap;
```

```
HashMap<Integer, String> studentNamesById = new HashMap<>();
```

4. Collection: HashMap

데이터 저장하기: put()

value자료형 put(key자료형 key, value자료형 value);

- 지정한 key에 해당하는 value를 HashMap에 저장한다.
- 동일한 key가 이미 존재하는 경우 기존 value를 새로운 value로 교체한다.
- 해당 key에 이전에 매핑되어 있던 value를 반환한다.
- 해당 key에 매핑되어 있던 value가 없는 경우 null을 반환한다.

```
studentNamesById.put(1001, "Alice");  
studentNamesById.put(1002, "Alice");  
studentNamesById.put(1002, "Bobby");
```

```
{ } studentNamesById = {HashMap@854}  
{ } {Integer@860} 1001 -> "Alice"
```

```
{ } studentNamesById = {HashMap@854}  
{ } {Integer@860} 1001 -> "Alice"  
{ } {Integer@865} 1002 -> "Alice"
```

```
{ } studentNamesById = {HashMap@854}  
{ } {Integer@860} 1001 -> "Alice"  
{ } {Integer@865} 1002 -> "Bobby"
```

4. Collection: HashMap

value 조회하기: get()

value자료형 get(key자료형 key);

- 지정한 key에 매핑되는 value를 반환한다.
- 해당 key가 존재하지 않는 경우 null을 반환한다.

```
String selectedStudentName = studentNamesById.get(1002);
```

```
{ } studentNamesById = {HashMap@854}  
  { } {Integer@860} 1001 -> "Alice"  
  { } {Integer@865} 1002 -> "Bobby"  
  { } selectedStudentName = "Bobby"
```

4. Collection: HashMap

key 존재 여부 확인하기: `containsKey()`

```
boolean containsKey(key자료형 key);
```

- 지정한 key가 HashMap에 존재하는지 여부를 반환한다.
- `get()`은 key가 존재하지 않거나 해당 key의 value가 null인 경우 null을 반환할 수 있다.
- `containsKey()`는 매핑된 value와 무관하게 key의 존재 여부만을 기준으로 판단한다.

```
boolean hasStudentId = studentNamesById.containsKey(1003);
```

```
{ } studentNamesById = {HashMap@854}  
{ } {Integer@860} 1001 -> "Alice"  
{ } {Integer@865} 1002 -> "Bobby"  
1 hasStudentId = false
```

4. Collection: HashMap

데이터 제거하기: remove()

```
value자료형 remove(key자료형 key);
```

- 지정한 key에 해당하는 key-value 쌍을 HashMap에서 제거한다.
- 제거된 key에 매핑되어 있던 value를 반환한다.
- 해당 key가 존재하지 않는 경우 null을 반환한다.

```
studentNamesById.remove(1001);
```

```
{ } studentNamesById = {HashMap@854}  
{ } {Integer@865} 1002 -> "Bobby"
```

4. Collection: HashMap

데이터 개수 확인하기: size()

```
int size();
```

- HashMap에 현재 저장된 key-value 쌍의 개수를 반환한다.
 - 저장 가능한 key-value 쌍의 개수가 아니라 현재 저장된 key-value 쌍의 개수를 반환한다.

```
int studentCount = studentNamesById.size();
```

```
{ } studentNamesById = {HashMap@854}  
{ } {Integer@865} 1002 -> "Bobby"  
1 studentCount = 1
```

[다양한 HashMap 메서드 보기 -> Java 17 API Specification]

다양한 컬렉션

- Java에는 ArrayList, HashSet, HashMap 외에도 다양한 목적을 가진 컬렉션들이 존재한다.
- 각 컬렉션은 저장 방식, 접근 방법, 사용 목적이 서로 다르다.
- 이 외에 대표적인 컬렉션들은 다음과 같다.
 - ArrayDeque: 양쪽 끝에서 삽입·삭제할 수 있어 스택이나 큐처럼 사용할 수 있는 컬렉션
 - Queue: 선입선출(FIFO) 구조로 사용할 수 있는 컬렉션 인터페이스
 - LinkedList: 노드 기반 구조로 앞뒤 요소의 추가·삭제가 상대적으로 빈번한 경우에 적합
- 상황에 맞는 컬렉션을 선택해 사용하는 것이 중요하다.

[다양한 컬렉션 보기 -> Java 17 API Specification]

Thank you

Wishing you an enjoyable and meaningful learning experience.
May you grow into an engineer who learns with joy and shares with others.

For inquiries, contact: potterLim0808@gmail.com

